

TECHNICAL DOCUMENTATION

WebGpuWater — API Reference

The public scripting surface, symbol by symbol: WaterVolume, the query seam, components, ScriptableObjects, and key shader uniforms.

Package com.abstractocclusion.webgpuwater · **Version** 1.0.0

Engine Unity 2022.2+ / URP 12+ · Desktop · WebGPU/WebGL · Mobile

Generated 20 July 2026 · A modern URP adaptation of Evan Wallace's WebGL Water (MIT)

Contents

API Reference	3
Enums	3
WaterVolume	4
Height-query seam	5
Components (MonoBehaviour)	6
ScriptableObjects	8
Shader / material property reference	8

API Reference

All runtime types live in the single namespace `AbstractOcclusion.WebGpuWater` (assembly `AbstractOcclusion.WebGpuWater` ; editor code in `AbstractOcclusion.WebGpuWater.Editor`). The public surface is deliberately small: inspector tuning and builder wiring are *not* public fields; runtime-scriptable state is exposed as properties, and gameplay logic goes through `WaterVolume` and a handful of components. This reference lists every public type and member intended for scripting.

```
using AbstractOcclusion.WebGpuWater;
```

Enums

Enum	Members	Notes
<code>WaterVolume.WaterBodyType</code>	<code>Pond</code> , <code>Lake</code> , <code>Ocean</code>	Advisory — drives inspector defaults, not runtime paths.
<code>WaterVolume.RippleQuality</code>	<code>Low</code> , <code>Medium</code> , <code>High</code> , <code>Ultra</code>	Interactive ripple grid quality.
<code>WaterVolume.ObjectInteraction</code>	<code>MouseLikeDrops</code> , <code>FootprintDelta</code>	How floating objects disturb the surface.
<code>WaterVolume.ReflectionMode</code>	<code>SkyOnly</code> , <code>SSR</code> , <code>Planar</code>	Retired; kept for scene migration. Runtime uses independent reflection toggles.
<code>WaterVolume.EnvironmentSource</code>	<code>ProceduralSky</code> , <code>UrpProbe</code>	Retired; kept for scene migration.
<code>WaterQueryFields</code>	<code>Height = 1</code> , <code>Normal = 2</code> , <code>Velocity = 4</code> , <code>HeightNormalVelocity = Height\ Normal\ Velocity</code>	[Flags] ; select which fields a query fills.
<code>WaterVolume.ChunkFootprint</code>	<code>None</code> , <code>Box</code> , <code>Sphere</code> , <code>Mesh</code>	Finite-water chunk shape.
<code>WaterFoamParticles.FoamRenderMode</code>	<code>ScreenSpaceDensity</code> , <code>Quads</code>	Foam-particle rendering path.
<code>WaterQuality.Selection</code>	<code>Auto</code> , <code>ForceLow</code> , <code>ForceMedium</code> , <code>ForceHigh</code>	Tier selection strategy.
<code>WaterQuality.UnderwaterMode</code>	<code>Off</code> , <code>Simple</code> , <code>Full</code>	Underwater-fog cost mode per tier.
<code>JerlovWaterType</code>	<code>OceanI</code> , <code>OceanIA</code> , <code>OceanIB</code> , <code>OceanII</code> , <code>OceanIII</code> , <code>Coastal1C</code> , <code>Coastal3C</code> , <code>Coastal5C</code> , <code>Coastal7C</code> , <code>Coastal9C</code> , <code>Legacy</code>	Physically-derived water-colour presets.

WaterVolume

The central body component (`MonoBehaviour`, `ISerializationCallbackReceiver`, `IWaterHeightSampler`), authored as a `partial class` . One per body of water.

Static members

```
static WaterVolume Primary { get; }           // global fallback body
static WaterVolume Resolve();                 // Primary, else a scene search
static WaterVolume BodyContaining(Vector3 worldPoint); // the body whose footprint
holds the point

static bool TrySampleHeightAt(Vector3 worldPos, out float worldY);
static bool IsSubmergedAt(Vector3 worldPos);
static bool TrySpawnRippleAt(Vector3 worldPos, float radius, float strength);
static bool TrySphereInteractionAt(Vector3 worldPos, Vector3 worldStep, float radius,
float strength);
static bool SampleHeightAcrossBodies(Vector3 worldPoint, out WaterSample sample, float
minimumLength = 0f);
```

Instance methods — queries

```
bool TryGetWaterHeight(float worldX, float worldZ, out float height);
bool TryGetSurface(float worldX, float worldZ, out float height, out Vector2 flow);
bool TryGetAnalyticWaterline(float worldX, float worldZ, out float height); // valid
from frame 0
bool TrySampleHeight(Vector3 worldPos, out float worldY);
bool TrySampleSubmersion(Vector3 worldPoint, out float depthWorld, out Vector3 up, out
Vector3 worldFlow);
bool IsSubmerged(Vector3 worldPos);
bool TryRaycastSurface(Ray ray, out Vector3 worldHit);

// IWaterHeightSampler
bool SampleHeight(Vector3 worldPoint, out WaterSample sample, float minimumLength = 0f,
bool excludeInteractiveRipples = false);
void SampleHeights(int ownerHash, float minimumLength, IReadOnlyList<Vector3> points,
WaterSample[] results,
    WaterQueryFields fields = WaterQueryFields.HeightNormalVelocity, bool
excludeInteractiveRipples = false);
```

Instance methods — disturbance & maintenance

```
void AddRipple(float worldX, float worldZ, float radius, float strength);
void SpawnRipple(Vector3 worldPos, float radius, float strength);
void AddSphereInteraction(Vector3 worldPos, Vector3 worldStep, float radius, float
strength);
void RebakeBed(); // re-bake terrain bed depth
void RebakeShoreDepth(); // re-bake the shore depth / SDF field
void ToggleShoreDepthDebug();
void ToggleShoreSdfDebug();
```

Instance methods — GPU-consumer surface

```
void WriteBodyProps(MaterialPropertyBlock mpb);    // fill an MPB with this body's per-
body uniforms
void WriteSimFrameUniforms(ComputeShader cs);    // bind the volume + sim-window frame
to a compute shader
```

Runtime properties

```
bool   Foam           { get; set; }
bool   WindWaves      { get; set; }
bool   WaterFog       { get; set; }
float  RippleStrength { get; set; }
float  RippleRadius   { get; set; }
float  TimeScale       { get; set; }
bool   EnableCulling  { get; set; }
bool   IsPrimary       { get; set; }
WaterQuality Quality { get; set; }

// Read-only surface / sim state (for advanced consumers)
RenderTexture SimStateTexture { get; }
RenderTexture FoamMaskTexture { get; }
int    SimResolution      { get; }
bool   IsSimulating       { get; }
bool   IsVisibleToCamera  { get; }
float  SimTexelWorldArea  { get; }
Bounds SimWorldBounds     { get; }
bool   IsWindowed         { get; }
Vector3 SimWindowCenter   { get; }
Vector3 SimWindowHalfExtent { get; }
```

Height-query seam

IWaterHeightSampler

The allocation-free height-sampling contract `WaterVolume` implements (see its two `SampleHeight` / `SampleHeights` overloads above).

WaterSample (struct)

```
public struct WaterSample
{
    public float   Height;    // world-space surface Y at the query point
    public Vector3 Normal;    // surface normal
    public Vector3 Velocity;  // surface flow / velocity
    public bool    Valid;     // false outside the body's footprint
}
```

WaterHeightQuery (sealed)

Per-owner reusable result buffers (main-thread only) — buoyancy uses this to avoid per-frame allocation.

```
WaterSample[] RentResults(int ownerId, int
count); // grow-on-demand, keyed on a stable id
void Release(int ownerId);
```

Components (MonoBehaviour)

WaterBuoyancy [RequireComponent(typeof(Rigidbody))]

Multi-point buoyancy: lift, righting torque, drag, and wave drift. Add beside a `Collider`. Key inspector fields: `buoyancy`, `waterLinearDamping`, `waterAngularDamping`, `samplesPerAxis` (1-3), `floatRadiusScale`, `waveDriftStrength`, `verticalSettleDamping`, and the opt-in `objectWidth`, `maxBuoyancyForce`, `surfaceRelativeDrag`, `dragCoefficients`, `ignoreInteractiveRipples`, `drawDebugGizmos`.

WaterInteractable [RequireComponent(typeof(Renderer))]

Marks a `Renderer` that disturbs the surface.

```
static IReadOnlyList<WaterInteractable> Active { get; }
float displaceScale = 1f; // relative disturbance weight
bool reflectsWaves = false; // acts as a static reflector wall
float WaterlineY(float restY);
bool IsSubmerged(float waterlineY);
Renderer Renderer { get; }
```

WaterSphereInteractor [AddComponentMenu("AbstractOcclusion/Water/Water Sphere Interactor")]

Directional velocity-dipole wake (Kelvin V-wake) from a moving object. Fields: `centerOffset`, `radius` (0 = auto), `strength`, `maxStepDistance`, `wakeFadeSpeed`. Composition-only; drives `WaterVolume.TrySphereInteractionAt`.

WaterRippleEmitter

Scripted ripple emission into whichever body contains the emit point.

```
void Emit(); // fire one ripple on demand
// inspector: emitOnMove, requireNearSurface, surfaceBand, radius/strength
```

WaterProbe

Fires events as a watched point crosses the surface.

```
UnityEvent Submerged { get; }
UnityEvent Emerged { get; }
// plus IsSubmerged / Body accessors
```

WaterMembership [RequireComponent(typeof(Renderer))], [ExecuteAlways]

Associates a floating object with its containing body each frame so it is lit and driven by the correct water.

WaterSplash [RequireComponent(typeof(Rigidbody))]

Detects a body punching through the waterline and triggers `WaterSplashEmitter.EmitSplash(...)` + `AddRipple(...)`.

WaterSplashEmitter [DisallowMultipleComponent]

Entry-splash system: pop → stick → drift droplets and an optional flipbook crown. Unifies airborne spray onto the GPU particle path via `QueueSplashBurst(...)` when a `WaterFoamParticles` is present.

WaterFoamParticles [AddComponentMenu("AbstractOcclusion/Water/Water Foam Particles")]

Fully GPU-resident foam/spray particles. Key fields: `renderMode` (`FoamRenderMode`), `capacity`, `spawnThreshold`, `spawnRate`, `maxSpawnPerFrame`, `sprayChance`, `sprayLaunchSpeed`, `lifeRange`, `sizeRange`, `sizeHeroPower`, `spawnMaxDistance`, `gravity`, `flowDrift`, `windDriftSpeed`, `drag`, `crestRollSpeed`, `flipbookGrid`, `flipbookFps`, plus an optional `WaterFoamProfile`.

WaterExclusionVolume [ExecuteAlways]

Carves the water out of an oriented box.

```
static IReadOnlyList<WaterExclusionVolume> Active { get; }
Vector3 size = Vector3.one;
bool    drawWaterWalls = true;
float   wallScatterBoost = 1.2f;    // [0.5..2]
Color   edgeColor = Color.black;
float   edgeIntensity;             // [0..1]
float   edgeSpread;               // [0.01..0.5]
// const int MaxVolumes = 4;
```

BoatController [RequireComponent(typeof(Rigidbody))], [AddComponentMenu("AbstractOcclusion/Water/Boat Controller")]

Example drivable hull: thrust eased by propeller wetness, speed-scaled steering, keel/lateral drag. Pairs with `WaterBuoyancy` + `WaterSphereInteractor`.

Debug & stress components

- `WaterMetricsOverlay` [AddComponentMenu("AbstractOcclusion/Water/Metrics Overlay")] — on-screen frame-time / FPS / floater-count / query-cost HUD.
- `WaterBuoyancyStressSpawner` [AddComponentMenu("AbstractOcclusion/Water/Buoyancy Stress Spawner")] — spawns a parametric grid of buoyant cubes.
- `WaterOceanFftDebugView` [AddComponentMenu("WebGpuWater/Debug/Ocean FFT Debug View")], [ExecuteAlways] — visualizes the FFT cascades.

Cameras

- `OrbitCamera` [RequireComponent(typeof(Camera))] — orbit a pivot; `Rotate(dx, dy)`, `Zoom(...)`, `SetView(...)`.
- `FlyCamera` [RequireComponent(typeof(Camera))] — free-fly (WASD/QE, right-mouse look, touch sticks).
- `SimpleFollowCamera` [AddComponentMenu("AbstractOcclusion/Water/Simple Follow Camera")] — yaw-only chase camera.

Showcase components

- `WaterShowcaseController` — cycles one live `WaterShowcaseStation` clone at a time (N/M or arrow keys).
- `WaterShowcaseStation` — a station template (name, description, orbit framing, sun override).
- `WaterShowcaseMover` — kinematic circular path for props.
- `WaterShowcaseDripper` `[RequireComponent(typeof(WaterRippleEmitter))]` — periodic `Emit()`.

ScriptableObjects

`WaterQuality` `[CreateAssetMenu(menuName = "AbstractOcclusion/WebGpuWater/Water Quality")]`

High / Medium / Low cost tiers with an automatic hardware probe.

```
Selection selection = Selection.Auto;    // Auto | ForceLow | ForceMedium | ForceHigh
Tier Resolve();                        // the resolved tier for this device/selection
static Tier Default { get; }           // the tier used when no asset is assigned
```

`Tier` (readonly struct) fields: `SimResolution`, `CausticResolution`, `GodRaySteps`, `GodRays`, `RichReflections`, `MaxWaveCount`, `RefineSteps`, `RenderScale`, `RealRefraction`, `MeshDetail`, `CausticInterval`, `ReadbackInterval`, `MaxFoamParticles`, `UnderwaterFog` (`UnderwaterMode`).

`WaterFoamProfile` `[CreateAssetMenu(menuName = "AbstractOcclusion/WebGpuWater/Water Foam Profile")]`

One master asset bundling four `drive`-gated sections shared across foam components:

`SharedLook` (tint, opacity, `particleAtlas`, `flipbookGrid`, `flipbookFps`, `sizeHeroPower`), `Ambient` (spawn/spray/life/size), `Veil` (density composite: opacity, `densityLowGain`, `densityHighGain`, breakup lace), and `Splash` (burst/crown shaping). A `null` profile leaves each component on its own inspector values.

`JerlovWaterTypes` (**static**)

```
static JerlovPreset Get(JerlovWaterType type); // physically-derived Extinction +
BodyColor preset
```

Shader / material property reference

The surface and companion shaders are driven almost entirely by uniforms published per body through a `MaterialPropertyBlock`, so there is normally no need to set these by hand — use the `WaterVolume` inspector or its runtime properties. The most useful published names, for advanced integration:

Property	Meaning
<code>_WaterTex</code> , <code>_WaterTexel</code>	Interactive ripple state RT (<code>RGBAFloat</code> : height, velocity, normal.xz) and its texel size
<code>_FoamMask</code>	Surface foam field
<code>_CausticTex</code>	Projected caustics RT (R = focus, G = occluder shadow)
<code>_BedTex</code> , <code>_UseBedDepth</code>	Baked bed-depth map + gate
<code>_Sky</code> , <code>_Tiles</code>	Per-body sky cubemap and pool tile texture
<code>_PlanarReflectionTex</code>	Planar mirror RT
<code>_LightDir</code> , <code>_SunColor</code>	Sun direction/colour (shared global)
<code>_VolumeCenter</code> , <code>_VolumeExtent</code> , <code>_VolumeRot</code>	The oriented volume frame
<code>_SimWindowed</code> , <code>_SimCenter</code> , <code>_SimExtent</code>	Camera-following sim-window frame
<code>_WaveA</code> , <code>_WaveB</code> , <code>_WaveCount</code> , <code>_WaveTime</code>	Packed wind-wave bank (per body)
<code>_OceanFftActive</code> , <code>_OceanFftDisplacement</code> , <code>_OceanFftNormal</code>	FFT-ocean gate + output textures
<code>_UsePlanar</code> , <code>_UseSSR</code> , <code>_RealRefraction</code>	Reflection/refraction mode
<code>_WaterFogColor</code> , <code>_WaterExtinction</code> , <code>_WaterFogDensity</code>	Beer-Lambert fog
<code>_GodRaySteps</code> , <code>_PeakedRefineSteps</code>	Quality-tier raymarch/refine counts

Property names are cached via `Shader.PropertyToID` in `WaterUniformPublisher` ; shader *declaration* names come from `WaterShaderNames` (`Root` = "AbstractOcclusion/WebGpuWater/"). Treat any uniform not listed here as internal.